Agile Track — Day 3 Assignment

Agile and Scrum — Foundations and Backlog Workshop

Solutions, prioritised backlogs and sample JIRA workflow

This solution follows the three sets and task order in the supplied Day 3 assignment. It provides product visions, epics, user stories, INVEST reviews, acceptance criteria, prioritisation, Planning Poker estimates, and team working agreements.

Implementation note: JIRA board screenshots/exports are not included because a live JIRA project was not created. The backlog tables and board configuration below are ready to enter into JIRA; capture a genuine screenshot after publishing them.

Scrum quick reference

Element	Solution / meaning
Product Owner	Maximises product value; owns and orders Product Backlog.
Scrum Master	Coaches Scrum, facilitates events and removes impediments; does not act as project manager assigning tasks.
Developers	Cross-functional team members who create a usable Increment and own the Sprint Backlog.
Events	Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective.
Artifacts	Product Backlog, Sprint Backlog, Increment; Product Goal, Sprint Goal and Definition of Done provide commitments.

Planning Poker & estimation guide

Use relative effort, complexity, uncertainty and risk—not hours. Fibonacci values used here: 1, 2, 3, 5, 8, 13. Team members reveal cards together; discuss outliers, clarify scope, and revote. A 13-point story is generally split before sprint planning.

Reference story used across examples: a small, clear, independently testable ‘view item details’ story = 3 points. Other estimates are relative to that anchor.

Set 1 — Online Grocery Delivery Application

1. Product vision and Scrum roles

Vision: For local supermarket customers who want convenient shopping, the Grocery Delivery App is a mobile/web service that lets them discover products, order groceries and receive deliveries at a selected address and time. Unlike phone-based ordering, it provides a transparent catalogue, secure checkout and order tracking.

Role	Responsibility	Anti-pattern
Product Owner	Prioritises customer value, clarifies requirements and accepts outcomes against criteria.	Acts as a secretary taking every stakeholder request without ordering value.
Scrum Master	Facilitates Scrum, coaches team and helps remove blockers.	Assigns work like a traditional project manager or dominates decisions.
Developers	Design, build, test and integrate a Done Increment; self-manage delivery.	Wait for detailed task assignments or throw quality/testing over the wall.

2. Epics and 12 user stories

Epic	ID	User story	MoSCoW	SP
Browse & Search	G1	As a shopper, I want to browse product categories so that I can discover groceries.	Must	3
Browse & Search	G2	As a shopper, I want to search by product name so that I can find an item quickly.	Must	3

Epic	ID	User story	MoSCoW	SP
Browse & Search	G3	As a shopper, I want to filter products by category/price so that I can narrow results.	Should	5
Browse & Search	G4	As a shopper, I want to view product details and availability so that I can decide what to buy.	Must	3
Cart & Checkout	G5	As a shopper, I want to add an available product to my cart so that I can prepare an order.	Must	3
Cart & Checkout	G6	As a shopper, I want to change item quantities so that my cart matches my needs.	Must	2
Cart & Checkout	G7	As a shopper, I want to remove an item from my cart so that I can correct my order.	Should	2
Cart & Checkout	G8	As a shopper, I want to see the subtotal and delivery fee so that I know the payable amount.	Must	5
Cart & Checkout	G9	As a shopper, I want to place an order using a supported payment method so that my purchase is confirmed.	Must	8
Delivery Tracking	G10	As a shopper, I want to select an available delivery slot so that delivery suits my schedule.	Must	5
Delivery Tracking	G11	As a shopper, I want to view order status so that I know the progress of delivery.	Must	3
Delivery Tracking	G12	As a shopper, I want to receive an order confirmation notification so that I have a record of purchase.	Should	2

3. INVEST review and rewrites

INVEST = Independent, Negotiable, Valuable, Estimable, Small, Testable. Review examples:

Original (weak)	Issue	Improved story
As a user, I want the entire grocery app so it works.	Too large; not estimable or small.	As a shopper, I want to search products by name so that I can find a product quickly.
As a shopper, I want the database and API built so the app is fast.	Technical-layer story; value is unclear.	As a shopper, I want product results to show name, price and availability so I can compare items.
As a shopper, I want checkout, payment, delivery and notifications.	Multiple outcomes; not small.	As a shopper, I want to confirm my cart and payment so that an order is created.

4. Given / When / Then acceptance criteria (5)

• **G2 Search:** Given products exist, When shopper enters ‘rice’, Then matching product names appear; Given no match, When search runs, Then an empty-state message appears.

• **G4 Details:** Given a product is selected, When details open, Then name, price and stock status are shown.

• **G5 Add cart:** Given a product is in stock, When shopper adds quantity 1, Then cart count and subtotal update.

• **G8 Price:** Given cart contains items, When cart is viewed, Then subtotal, delivery fee and total are displayed clearly.

• **G9 Negative payment:** Given payment is declined, When checkout is submitted, Then order is not marked paid, cart remains available and a retry message is shown.

5. Prioritisation and estimates

MoSCoW: Must = essential first release; Should = important but workaround exists; Could = desirable if capacity remains; Won’t = explicitly deferred. The table above assigns Must/Should. Could: product recommendations, favourites, coupons; Won’t: voice shopping and multi-store comparison in first release.

Total backlog: 46 story points. Use 3-point G4 as reference. Poker outlier example: G9 payment receives 5 vs 13; discuss payment gateway, failure handling and security scope; agree 8 after slicing the payment integration from advanced payment options.

6. JIRA setup, DoR and DoD

JIRA: Create Scrum project ‘Grocery Delivery’; create three epics; add G1–G12 as Stories; fields: priority, story points, acceptance criteria, epic link. Board columns: To Do → In Progress → In Review → Done. Sprint 1 candidates: G1, G2, G4, G5, G6 (14 points; adjust to team capacity).

Definition of Ready (DoR):

• Story has clear user/value statement and Product Owner priority.

• Acceptance criteria are agreed and testable.

• Dependencies and assumptions are identified.

• Story is small enough (normally ≤8 points) and estimable.

• Required UX/data/API details are available; team understands it.

• No unresolved blocker prevents starting.

Definition of Done (DoD):

• Code reviewed and merged to main branch.

• Unit and relevant integration tests pass.

• Acceptance criteria verified; negative paths tested.

• Security, validation and error handling checked.

• No critical/high defects remain; regression checks pass.

• Documentation/release notes updated and Increment is deployable.

Set 2 — Insurance Claims Portal

1. Product vision and Waterfall vs Agile

Vision: For policyholders who need to file and follow insurance claims, the self-service Claims Portal enables secure claim registration, evidence upload and status visibility, reducing paperwork and uncertainty while helping claims teams process complete submissions.

Waterfall	Agile
Sequential phases; requirements are baselined early and change is costly.	Short iterations; feedback and reprioritisation are built into delivery.
Testing/integration often later; risks may surface late.	Testing and integration occur continuously; risks are surfaced early.
Useful when scope is stable and compliance needs fixed documentation.	Better fit when customer journeys, rules and integrations need learning and frequent validation.

Why Agile: Claim rules, document requirements and customer experience can evolve; incremental releases allow policyholders/claims staff to validate workflows early. Compliance remains explicit through traceability, acceptance criteria, audit logs and a strong DoD.

2. Four epics and 13 vertical user stories

Epic	ID	User story	Priority	SP
Claim Registration	C1	As a policyholder, I want to sign in securely so that only I access my claims.	Must	3
Claim Registration	C2	As a policyholder, I want to choose a policy so that a claim is linked to the right coverage.	Must	3
Claim Registration	C3	As a policyholder, I want to enter incident date and description so that the insurer understands the event.	Must	5
Claim Registration	C4	As a policyholder, I want to submit a claim and receive a claim number so that it is registered.	Must	5
Document Upload	C5	As a policyholder, I want to upload a required document so that evidence is attached to my claim.	Must	5
Document Upload	C6	As a policyholder, I want to see upload validation errors so that I can correct invalid files.	Must	3
Document Upload	C7	As a policyholder, I want to view uploaded documents so that I can verify evidence submitted.	Should	3
Claim Tracking	C8	As a policyholder, I want to see the current claim status so that I know its progress.	Must	3
Claim Tracking	C9	As a policyholder, I want to see a status timeline so that I understand key claim events.	Should	5
Claim Tracking	C10	As a claims officer, I want to update claim status so that the portal reflects processing progress.	Must	5
Settlement Notifications	C11	As a policyholder, I want an email when claim status changes so that I receive timely updates.	Should	5
Settlement Notifications	C12	As a policyholder, I want a settlement decision notification so that I know the outcome.	Must	3
Settlement Notifications	C13	As a policyholder, I want to view settlement amount and explanation so that I understand payment.	Must	5

3. INVEST review and vertical splitting

Too-large story: ‘As a customer I want to register, upload all documents, track and settle a claim’. Split by user outcome—not by UI/API/database layers:

• Register claim with incident details and receive claim number (C3–C4).

• Upload one evidence file and receive validation feedback (C5–C6).

• View claim status/timeline (C8–C9).

• Receive status and settlement notifications (C11–C12).

Each slice is user-visible/testable and can be prioritised independently. Avoid stories such as ‘create claims table’ because they describe a technical layer, not customer value.

4. Claim Registration acceptance criteria

• **C2 Policy:** Given the customer has active policies, When policy selection opens, Then only eligible policies are selectable.

• **C3 Details:** Given a valid policy, When incident date and description are entered, Then required fields are validated and the draft can proceed.

• **C4 Submit:** Given required fields are valid, When Submit Claim is selected, Then a unique claim number is generated and confirmation is displayed.

• **Validation failure:** Given incident date is in the future or description is blank, When submission is attempted, Then field-level errors appear and no claim is registered.

• **Duplicate protection:** Given submission is processing, When Submit is clicked again, Then only one claim is created.

5. Planning Poker and backlog total

Use values 1, 2, 3, 5, 8, 13 and estimate relative to a 3-point reference story (C8: view status). Poker outlier: C5 upload may be 3 for a basic file vs 8 for virus scan, storage, size/type checks and retries; clarify scope, then agree 5 for the defined single-file upload.

Total: 58 story points (13 stories). The estimate is a team-relative forecast, not a promise or conversion to hours.

6. JIRA project, board and Scrum note

Project: ‘Claims Self-Service’. Epics: Claim Registration, Document Upload, Claim Tracking, Settlement Notifications. Workflow: To Do → In Progress → In Review → Done. Add story points, priority, acceptance criteria, epic link and attachment fields. Attach DoD and use a Sprint Goal such as ‘Policyholders can submit a valid claim and receive a claim reference’.

Roles: Product Owner orders backlog and clarifies value; Scrum Master coaches/facilitates; Developers deliver Increment. **Events:** Sprint Planning, Daily Scrum, Sprint Review, Retrospective. **Artifacts:** Product Backlog, Sprint Backlog, Increment; commitments are Product Goal, Sprint Goal and DoD.

DoD: reviewed code; tests pass; claim data access secured; validation and audit trail verified; acceptance criteria met; no critical defects; documentation updated; deployable Increment.

Set 3 — Hospital Appointment Booking Application

1. Vision and stakeholders

Vision: For patients who need convenient access to doctors, the Hospital Appointment App provides a reliable way to find a doctor, book/reschedule/cancel appointments and receive reminders, while helping hospital staff manage accurate schedules.

Stakeholder	Scrum mapping	Contribution
Patients	Customer / users	Search doctors, book and manage appointments; give feedback.
Hospital management	Customer / stakeholder	Business goals, policies and success measures.
Doctors	Customer / stakeholder	Availability, clinical schedule constraints.
Reception/scheduling staff	Customer / stakeholder	Operational rules and exception handling.
Product representative	Product Owner	Owns vision, backlog ordering and acceptance.
Facilitator/coach	Scrum Master	Supports Scrum, removes impediments.
Engineering, QA, UX	Developers	Build, test and deliver the product Increment.

2. Epics and 12 user stories

Epic	ID	User story	MoSCoW	SP
Booking	H1	As a patient, I want to search doctors by specialty so that I can find a suitable doctor.	Must	3
Booking	H2	As a patient, I want to view a doctor's available slots so that I can choose a convenient time.	Must	3
Booking	H3	As a patient, I want to book an available slot so that I can reserve an appointment.	Must	5
Booking	H4	As a patient, I want a booking confirmation with appointment details so that I can verify it.	Must	2
Rescheduling & Cancellation	H5	As a patient, I want to reschedule my appointment to an available slot so that I can change plans.	Must	5
Rescheduling & Cancellation	H6	As a patient, I want to cancel an appointment within policy so that the slot is released.	Must	3
Rescheduling & Cancellation	H7	As a patient, I want to see cancellation rules before cancelling so that I understand the impact.	Should	2
Reminders	H8	As a patient, I want a reminder before my appointment so that I do not forget it.	Must	3
Reminders	H9	As a patient, I want to choose reminder preferences so that notifications suit me.	Could	3
Doctor Schedule Management	H10	As scheduling staff, I want to set doctor availability so that patients see valid slots.	Must	5
Doctor Schedule Management	H11	As a doctor, I want to block unavailable periods so that appointments are not booked then.	Must	3
Doctor Schedule Management	H12	As scheduling staff, I want to view daily appointments so that I can manage clinic flow.	Should	5

3. INVEST review and rewrites

Weak story: ‘As a patient I want to book and manage all appointments’. It is too broad and not small. Split into H2 (view slots), H3 (reserve slot), H4 (confirmation), H5 (reschedule) and H6 (cancel).

Weak story: ‘As a user I want notifications and schedule management’. It combines actors and outcomes. Rewrite as H8 (patient reminder) and H10/H11 (staff/doctor schedule control). Check each story is independently valuable, negotiable, estimable, small and testable.

4. Booking and cancellation acceptance criteria

• **H2 Slots:** Given a doctor and date, When available slots are requested, Then only open slots are shown in the doctor's timezone.

• **H3 Booking:** Given a slot is available, When patient confirms booking, Then the slot is reserved and an appointment ID is shown.

• **Slot already taken:** Given another patient has just booked the selected slot, When this patient confirms, Then booking is rejected, no duplicate appointment is created and refreshed slots are offered.

• **H6 Cancellation:** Given an appointment is cancellable, When patient confirms cancellation, Then appointment status becomes Cancelled and slot is released.

• **Cancellation negative:** Given cancellation is outside the allowed window, When patient attempts it, Then cancellation is blocked and policy guidance is displayed.

5. Value vs effort, Poker and story points

Prioritise Must stories with high patient/operational value first: H2, H3, H4, H6, H10, H11, H8. Next: H5, H7, H12; then H9. Value-effort quick view: high value/low effort (H4, H6) first; high value/high effort (H3, H10) plan early; low value/low effort (H9) later; low value/high effort defer.

Total: 42 story points. Reference story: H4 confirmation (2 points) is a small, well-understood baseline; estimate other stories by relative complexity. Poker example: H3 booking gets 3 vs 8 due to race conditions and slot locking; discuss concurrency and duplicate prevention, then agree 5 for booking with slot reservation.

Story points vs hours: Points express relative size/complexity/uncertainty and are team-specific; hours are elapsed/effort time estimates. A 5-point story is not five hours, and velocity should not be used to compare teams.

6. JIRA and DoR / DoD checklists

Create Scrum project ‘Hospital Appointments’; epics: Booking, Rescheduling & Cancellation, Reminders, Doctor Schedule Management. Add H1–H12, link epics, assign priority and points. Workflow: To Do → In Progress → In Review → Done. Sprint goal example: ‘Patients can book and cancel an appointment safely’.

Definition of Ready — at least six:

• User/value statement is clear.

• Acceptance criteria are written and testable.

• Priority is set by Product Owner.

• Dependencies, schedule rules and assumptions are identified.

• Story is estimable and appropriately sized.

• UX/API/data needs are understood.

• No blocker prevents implementation.

Definition of Done — at least six:

• Code reviewed and merged.

• Automated unit/integration tests pass.

• Booking race/slot conflict cases tested.

• Acceptance criteria and cancellation policy verified.

• Security/privacy checks pass; patient data protected.

• No critical defects; regression suite passes.

• Release notes/documentation updated.

• Increment is deployable and Product Owner can review.

JIRA board and submission checklist

• Create project and board; add all epics and stories from the backlog tables.

• Set columns To Do, In Progress, In Review, Done and map statuses correctly.

• Set story points, priority, acceptance criteria and epic link for every story.

• Add a DoR/DoD page or attachment and link it to the project.

• Take a genuine screenshot/export of the board and backlog; include it with this solution.

• Review story sizes and sprint capacity with the actual team before committing.

Note: The estimates and priorities here are worked examples. A real Scrum team should discuss and agree on them collaboratively; estimates can change as the team learns.